

How are Numbers Stored in java

* Keywords

→ Some keywords are used by java which we cannot use while naming our variables

eg: public, class, static, void, ... etc (total 68 keywords)

→ keywords are reserved words by java.

* Comments

→ If we use comments in our code, java will ignore our comment

single line comment: → // Comment here

Multi line comment: → /*

Comment here

*/

* Negative Numbers

byte b = -42;

→ 8 bit: _____

↳ -128 to +127

+42: 0 0 1 0 1 0 1 0

~~1 1 1 1 1 1 1 1~~

128 64 32 16 8 4 2 1

00101010 = 42

(in binary)

byte b = 42;

↳ 00101010
↳ stored in memory

System.out.println(b);

→ 42

byte b = -42; [2's complement]

ignore sign = 42 = 00101010

1's complement → 11010101

(reverses zero with 1)

2nd complement → 1's complement + 1 → 11010101 + 1 → 11010110

11010110

this is stored in memory

byte b = -42;

→ 11010110

stored in memory

system.out.println(b); ←

11010110

MSB

(Most significant bit)

LSB

(Least significant bit)

MSB = 1 → -ve Number

MSB = 0 → +ve Number

byte: 0 1 1 1 1 1 1 1 : 127
128 64 32 16 8 4 2 1

-128 to 127

it cannot be 1 in case of the number

11010110

system.out.println(b);

MSB = 1 → -ve no.

1's C → 00101001

2's C → 00101010 → 42

but since MSB = 1

∴ it is -ve

∴ it'll print = -42

why need of 2's C:

byte b = 0;

0 0 0 0 0 0 0 0

because : byte b = -0; → wrong

1's → 11111111

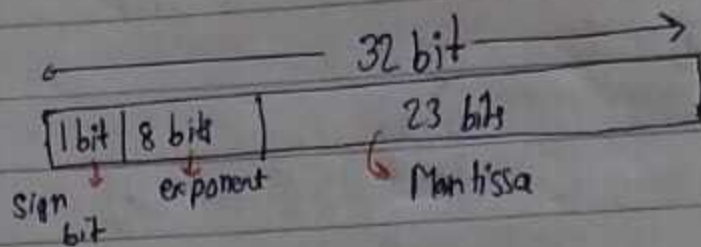
-0 = 0

2's → 10000000 → 0

* How are floating numbers stored.

float f = 8.125f;

float f = 0.7f;



f = 8.125

1) 1000.001
find binary

$$0.125 \times 2 = 0.25 \rightarrow 0$$

$$0.25 \times 2 = 0.5 \rightarrow 0$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

2) Make it in the form of

$$(1.x \dots) \times 2^{\text{exp}}$$

$$1.\underline{000001} \times 2^3$$

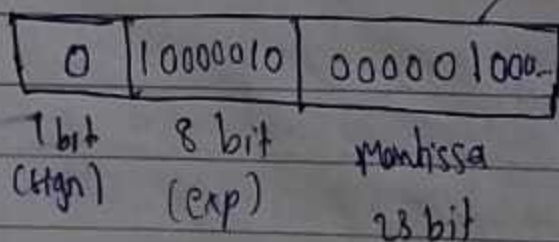
mantissa

3) Add Bias to the exponent
for float bias = 127

$$3 + 127 = 130 = 10000010$$

2	130	0
2	65	0
2	32	1
2	16	0
2	8	0
2	4	0
2	2	0
	1	0

4) Place value in memory



System.out.println(f);

How it retrieves $\rightarrow (-1)^{\text{sign}} \times (1 + \text{Mantissa}) \times 2^{\text{exp} - \text{bias}}$
 $= (-1)^0 \times (1 + 000001) \times 2^{130 - 127}$

$$\begin{aligned}
 &= 1 \times \left(1 + \frac{1}{64}\right) \times 2^3 \\
 &= 1 \times \left(1 + 2^{-6}\right) \times 8 \\
 &= \left(1 + \frac{1}{64}\right) \times 8 \\
 &= \frac{65}{8} = 8.125
 \end{aligned}$$

111.274

$$\frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^6} + \frac{1}{2^7} + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{10}}$$

but for float $f = 0.7f$;

1) Express to binary $\Rightarrow 0.7$

$0.1011001100110...$

$$0.7 \times 2 = 1.4 \rightarrow 1$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

$$0.6 \times 2 = 1.2 \rightarrow 1$$

$$0.2 \times 2 = 0.4 \rightarrow 0$$

$$0.4 \times 2 = 0.8 \rightarrow 0$$

$$0.8 \times 2 = 1.6 \rightarrow 1$$

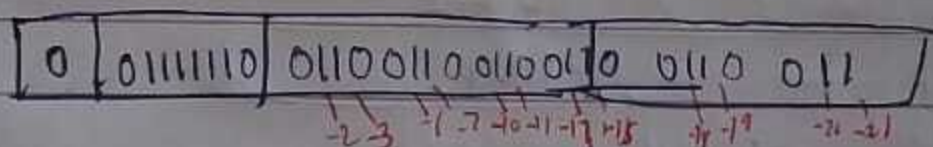
$$2) = (1.777) \times 2^{\text{exp}}$$

$$(1.01100110...) \times 2^{-1}$$

3) Add bias $\Rightarrow 127$

$$-1 + 127 = 126 \stackrel{+}{=} 01111110$$

4) Express it in memory



System.out.println(f);

$$\rightarrow (-1)^s \times (1 + \text{Mantissa}) \times 2^{e-b}$$

$$= (-1)^0 \times \left(1 + \frac{1}{2}\right) \times 2^{126-127}$$

$$= \left(1 + \frac{1}{2}\right) \times \frac{1}{2}$$

$$= \frac{1}{2} \left(1 + \frac{1}{2^2} + \frac{1}{2^3} + \frac{1}{2^4} + \frac{1}{2^5} + \frac{1}{2^6} + \frac{1}{2^7} + \frac{1}{2^8} + \frac{1}{2^9} + \frac{1}{2^{10}} + \frac{1}{2^{11}} + \frac{1}{2^{12}} + \frac{1}{2^{13}} + \frac{1}{2^{14}} + \frac{1}{2^{15}} + \frac{1}{2^{16}} + \frac{1}{2^{17}} + \frac{1}{2^{18}} + \frac{1}{2^{19}} + \frac{1}{2^{20}} + \frac{1}{2^{21}} + \frac{1}{2^{22}}\right)$$

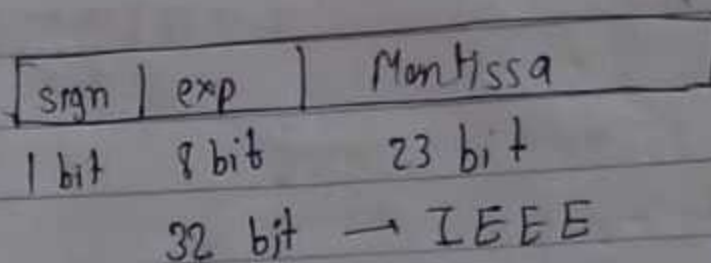
$$= 1.399999761581420891438/2$$

$$= 0.6999998807907104421975$$

* Bias ??

$$-127 + 127 = 0$$

$$\text{Bias} = 127 \approx$$



$$0.7f = 2^{-1} = 2^{126} \quad \text{negative}$$

$$\text{exp} = 8 \text{ bit} = 0 - 255 \text{ char}$$

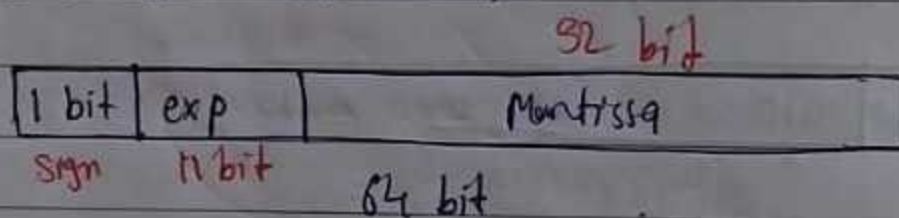
(Range of exp) $2^8 = 256$

$$\hookrightarrow \text{bias} = (2^{8-1} - 1) = (2^7 - 1) = 127$$

$$\boxed{\text{bias} = 2^{\text{exp bit} - 1} - 1} \quad \text{* Imp}$$

* What about double

$$\text{double } d = 32.4156;$$



& same as float procedure

$$\text{but here bias} = (2^{11-1} - 1) = (2^{10} - 1) = 1023.$$

float & double → estimated value

→ but in java we have BigDecimal gives exact value
but we'll study this in non primitive data types.